

Proyecto 01 - Creación de un RAG

Sección 1 - Reporte

Dominio y tamaño del corpus: Para este proyecto, elegí el tema de distintos animales marinos. Tomé artículos de Wikipedia de algunos de los animales marinos más comunes y generé 7 fuentes en PDF para que el RAG tomé como base de conocimiento. (Tortugas marinas, tiburones, medusas, pulpos, delfines, ballenas y cangrejos).

Dada la naturaleza de las fuentes (Wikipedia), no todas las fuentes de animales tienen los mismos campos de información. Algunos artículos pueden tener, por ejemplo, datos sobre la dieta de los animales. Consideré esta diferencia entre fuentes para corroborar que el RAG no alucine o genere respuestas de cosas de las que no tiene noción.

El corpus son los 7 PDFs generados, los cuales generaron un índice de 17 chunks vectorizados.

Algunos datos técnicos:

- Modelo de embeddings: text-embedding-001
- Modelo de IA: gemini-3.8-flash

Estrategia de Partición: El chunk size elegido fue de 300, y el overlap de 50. Con 300 unidades puede desarrollarse una idea simple, que puede caber en un párrafo. El overlap con valor 50 evita que oraciones importantes puedan quedar cortadas. De esta forma, se mejora la calidad de la recuperación.

Mecanismo de abstención: Para tener la certeza de que el modelo se abstenga de responder cuando no tenga suficiente información, usé 2 estrategias.

La primera fue una validación de existencia de documentos/fuentes para generar respuestas. La función `generar_respuesta` primero corrobora si hay documentos. En caso de no haberlos, devuelve un texto especificando que no se tiene evidencia para generar una respuesta.

La segunda fue una restricción en el prompt. Simplemente le pedí al modelo que generara respuestas basadas en la información disponible. De lo contrario, no inventar respuestas y avisar al usuario que la evidencia no es suficiente para responder las preguntas.

Roles de cada servicio/plataforma en mi arquitectura:

1. Chroma DB: funge como mi almacén de datos persistente en el disco. Guarda los vectores de Google AI y sus metadatos (en este caso, el origen). También hace

las operaciones matemáticas que permiten encontrar los textos más cercanos a la pregunta del usuario.

2. Google AI: Procesa las tareas de comprensión de texto. Convierte los textos en vectores. Lee el contexto recuperado para generar respuestas fluidas. Cita las fuentes usadas para generar sus respuestas.

3. Streamlit: Sirve como plataforma con interfaz gráfica para que el usuario pueda usar el modelo de manera más sencilla. Permite la carga de archivos y hacer preguntas en una interfaz familiar.

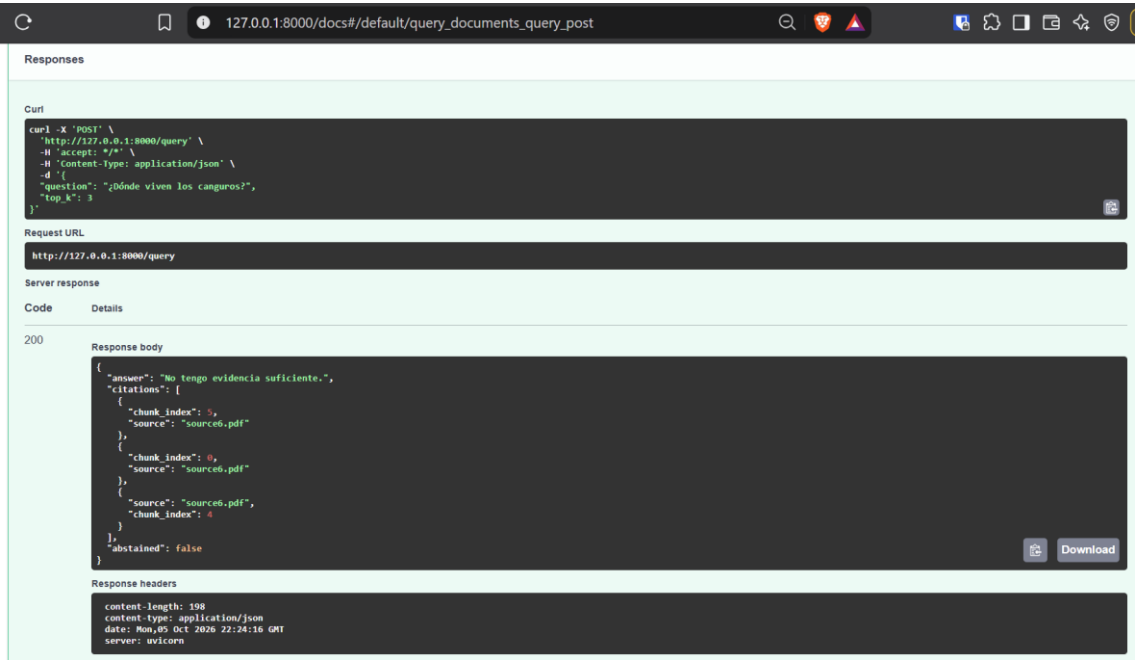
Sección 2 – Evidencias

Para esta sección hice algunas preguntas para evaluar el modelo. A continuación, muestro las respuestas generadas.

2.1 *Pregunta imposible:*

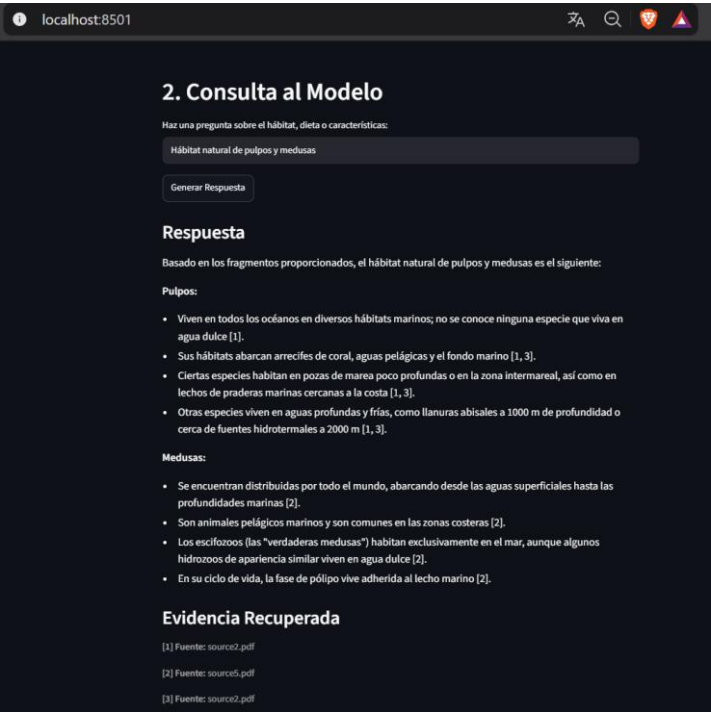


2.2 *Pregunta imposible – backend view:*



2.3 Preguntas dentro del dominio;

Ejemplo 1:



Ejemplo 2:



Ejemplo 3: (Mixto – sabe una respuesta, y otra la desconoce)

Frontend:

